EE3621: Digital Signal Processing

Lecture 27: Matched-Z IIR Design Example and the Adaptive Filter Framework (Wiener Solution & LMS)

(III B.Tech EEE)

Comprehensive Lecture Notes & Faculty Textbook Reference

Lecture Plan & Objectives (40 Minutes Breakdown)

- **00:00 – 05:00 (5 mins):** Recap: analog-to-digital IIR conversion and the pole-mapping $z = e^{sT}$; why zeros "at infinity" need special care.
- **05:00 – 20:00 (15 mins):** Worked Design (Example 27.1): digital filter from $H_a(s) = \frac{2}{(s+1)(s+2)}$ with $T = 0.1\,\mathrm{s}$ — pole/zero bookkeeping, pole mapping, zeros placed at $z = -1$, DC-gain matching.
- **20:00 – 27:00 (7 mins):** The Adaptive Filter Framework: input $x[n]$, desired $d[n]$, output $y[n]$, error $e[n]$, update algorithm (block diagram).
- **27:00 – 35:00 (8 mins):** Mathematics: input/weight vectors, MSE cost $J(\mathbf{w})$, normal equations (Wiener solution), and the Widrow–Hoff LMS update.
- **35:00 – 40:00 (5 mins):** Worked LMS iteration (Example 27.2), pitfalls, and a teaser for Lecture 28 (why this is already machine learning).

---

1 1. LEARNING OBJECTIVES

By the end of this lecture, students will be able to:

1. **Count** the poles, finite zeros and zeros at infinity of an analog prototype (N, M, $N-M$).
2. **Map** analog poles to the z-plane using $p_k \mapsto e^{p_k T}$ and **place** the $N - M$ missing zeros at $z = -1$.
3. **Match** the DC gain of the digital filter to the analog prototype by computing a scale factor K_d.
4. **Draw** the adaptive filter block diagram and identify $x[n]$, $d[n]$, $y[n]$, $e[n]$ and the update block.
5. **Write** the MSE cost $J(\mathbf{w}) = E[d^2] - 2\mathbf{w}^T\mathbf{p} + \mathbf{w}^T\mathbf{R}\mathbf{w}$ and its minimizer $\mathbf{w}^* = \mathbf{R}^{-1}\mathbf{p}$.
6. **Derive** the LMS recursion $\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} + \mu\, e[n]\, \mathbf{x}[n]$ (tap-by-tap: $w_k^{(i+1)} = w_k^{(i)} + \mu\, e[n]\, x[n-k]$) from an instantaneous gradient estimate.
7. **Disambiguate** clearly between discrete-time index n, filter tap index k, and adaptation iteration step i.

---

2 2. IIR DESIGN BY POLE MAPPING WITH ZEROS AT $z = -1$

2.1 2.1 Idea

A strictly proper analog filter $H_a(s)$ with N poles and $M < N$ finite zeros has $N - M$ zeros at $s = \infty$. Under the mapping $z = e^{sT}$:

- each pole p_k maps to $z_k = e^{p_k T}$ (stable $\Rightarrow |z_k| < 1$);
- each finite zero q_k maps to $e^{q_k T}$;
- the $N - M$ zeros at $s \to \infty$ correspond to the highest analog frequency, $\Omega = \pi/T$, i.e. to $\omega = \pi$, i.e. to $z = -1$. We therefore **insert** $(N - M)$ **zeros at** $z = -1$, i.e. factors $(1 + z^{-1})^{N-M}$.

Because pole/zero mapping does not preserve gain, the result is scaled by a constant K_d so that the DC gains agree: $H(1) = H_a(0)$.

2.2 2.2 Procedure

1. Identify poles (N), finite zeros (M), zeros at infinity $(N - M)$.
2. Map all poles and finite zeros with $e^{(\cdot)T}$.
3. Add $N - M$ zeros at $z = -1$ to form the unscaled filter $H''(z)$.
4. Compute $K_d = H_a(0)/H''(1)$ and set $H(z) = K_d\, H''(z)$.

3 3. WORKED EXAMPLE 27.1 – SECOND-ORDER LOW-PASS

Problem: Design a digital filter from the 2nd-order analog lowpass

$$H_a(s) = \frac{2}{(s+1)(s+2)}, \qquad T = 0.1 \text{ s}. \tag{1}$$

Step 1: Poles, zeros and degrees.

- Poles: $p_1 = -1$, $p_2 = -2 \implies N = 2$.
- Finite zeros: none $(M = 0)$.
- Zeros at infinity: $N - M = 2 - 0 = 2$.

Step 2: Map poles; add zeros at $z = -1$.

$$e^{p_1 T} = e^{-1 \times 0.1} = e^{-0.1} \approx 0.9048 \tag{2}$$

$$e^{p_2 T} = e^{-2 \times 0.1} = e^{-0.2} \approx 0.8187 \tag{3}$$

Two zeros are added at $z = -1$, giving $(1 + z^{-1})^2$. The unscaled digital transfer function is

$$H''(z) = \frac{(1 + z^{-1})^2}{(1 - 0.9048\, z^{-1})(1 - 0.8187\, z^{-1})}. \tag{4}$$

Step 3: Match the DC gain. For the analog filter at $s = 0$:

$$H_a(0) = \frac{2}{(0+1)(0+2)} = 1. \tag{5}$$

For the unscaled digital filter at $z = 1$ ($z^{-1} = 1$):

$$H''(1) = \frac{(1+1)^2}{(1-0.9048)(1-0.8187)} = \frac{4}{(0.0952)(0.1813)} \approx \frac{4}{0.01726} \approx 231.75. \tag{6}$$

To force $H(1) = 1$:

$$K_d = \frac{H_a(0)}{H''(1)} = \frac{1}{231.75} \approx 0.004315. \tag{7}$$

Result:

$$\boxed{H(z) = \frac{0.004315\,(1+z^{-1})^2}{(1-0.9048\,z^{-1})(1-0.8187\,z^{-1})}} \tag{8}$$

Difference equation. Expanding the denominator, $(1 - 0.9048z^{-1})(1 - 0.8187z^{-1}) = 1 - 1.7235\,z^{-1} + 0.7408\,z^{-2}$, and the numerator $0.004315(1 + 2z^{-1} + z^{-2})$, so

$$y[n] = 1.7235\,y[n-1] - 0.7408\,y[n-2] + 0.004315\big(x[n] + 2x[n-1] + x[n-2]\big). \tag{9}$$

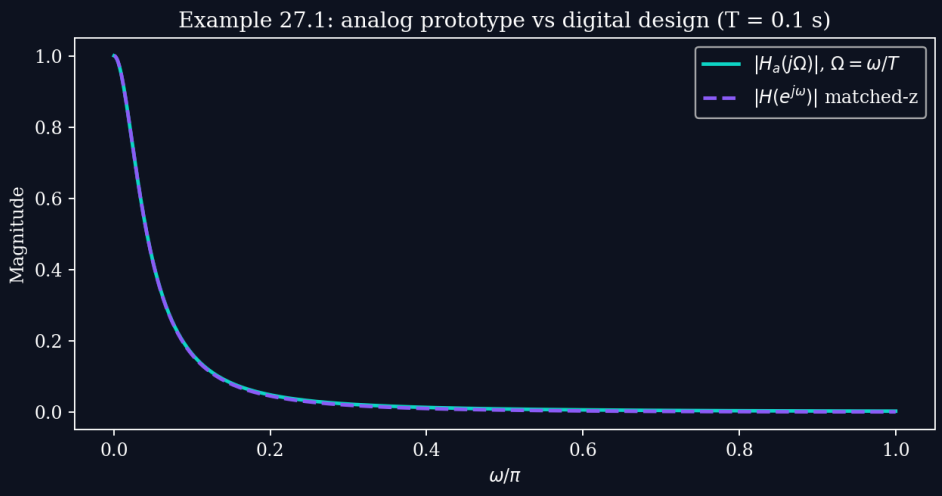


Figure 1: Magnitude of the analog prototype versus the digital filter of Example 27.1. The double zero at $z = -1$ drives the digital response to zero at $\omega = \pi$ whereas the analog response is only small (≈ 0.002 at $\Omega = \pi/T$).

Check: $|H(e^{j0})| \approx 1.000$, and $|H(e^{j\pi})| = 0$ because of the zeros at $z = -1$.

3.1 3.1 Design remarks

- The pole mapping is exact for the poles but the *frequency response* is only approximate: this is not a bilinear transform, and aliasing is not eliminated. It works well when $|H_a(j\Omega)|$ is already small near $\Omega = \pi/T$ (here $T = 0.1$ gives $\pi/T \approx 31.4$ rad/s, far beyond the poles at 1 and 2 rad/s).
- Choose T small compared with the inverse of the fastest pole; if T were 1 s the poles would map to 0.37 and 0.14 and the match would deteriorate.
- Always *match the gain at a chosen reference frequency*; DC is natural for lowpass filters.

---

4 4. THE ADAPTIVE FILTER FRAMEWORK

Everything so far (Lectures 15–27) designed **fixed** coefficients from a known specification. An **adaptive filter** instead *learns* its coefficients from data, by comparing its output with a desired signal.

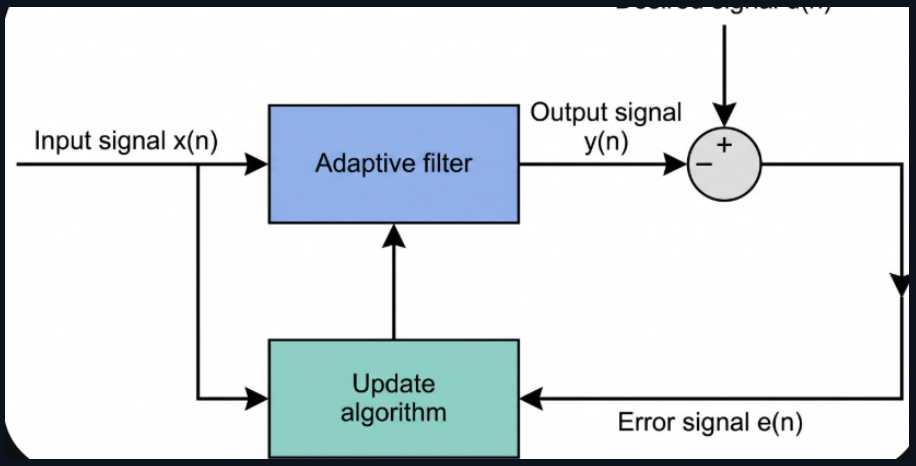


Figure 2: Adaptive filter: the filter output $y[n]$ is subtracted from the desired signal $d[n]$; the error $e[n]$ and the input $x[n]$ drive the update algorithm that adjusts the filter coefficients.

Variable / Signal	Role in Framework	Domain / Index Range
n	Discrete-time sample index	$n \in \{0, 1, 2, \dots\}$
k	Filter tap index (delay stage)	$k \in \{0, 1, \dots, M-1\}$
i	Adaptation iteration step	$i \in \{0, 1, 2, \dots\}$
$x[n]$	Input reference signal	Sample arriving at time n
$d[n]$	Desired (target) response	Reference signal at time n
$y^{(i)}[n]$	Adaptive filter output	Transversal sum at time n for iteration i
$e^{(i)}[n] = d[n] - y^{(i)}[n]$	Instantaneous estimation error	Drives coefficient update at iteration i
$w_k^{(i)}$	Weight of tap k at iteration i	Adaptive filter coefficient

4.1 4.1 Vector Notation & Index Disambiguation

In classical DSP textbooks, the symbol n is frequently overloaded to represent both physical discrete time ($x[n]$) and the optimization iteration counter ($w_k[n]$). To eliminate confusion between the **tap position** (k), the **time sample** (n), and the **algorithm iteration** (i):

- $k \in \{0, 1, \dots, M-1\}$ is the **spatial / tap index** along the FIR tapped delay line.
- n is the **discrete-time sample index** of the continuous signal streams $x[n]$ and $d[n]$.
- i is the **adaptation iteration index** ($i = 0, 1, 2, \dots$). In sample-by-sample LMS, iteration i processes sample $n = i$.

At discrete time n, the M-tap input delay line stores the current and past input samples:

$$\mathbf{x}[n] = \big[x[n],\ x[n-1],\ \dots,\ x[n-M+1]\big]^T \quad (M \times 1 \text{ input vector}) \tag{10}$$

The adaptive filter coefficients at iteration i form the weight vector:

$$\mathbf{w}^{(i)} = \big[w_0^{(i)},\ w_1^{(i)},\ \dots,\ w_{M-1}^{(i)}\big]^T \quad (M \times 1 \text{ weight vector}) \tag{11}$$

Filter output at sample n using weights of iteration i:

$$y^{(i)}[n] = \left(\mathbf{w}^{(i)}\right)^T \mathbf{x}[n] = \sum_{k=0}^{M-1} w_k^{(i)}\, x[n-k]. \tag{12}$$

Notice the clean separation: the sum is taken over the tap index k, multiplying the k-th tap weight $w_k^{(i)}$ by the delayed input sample $x[n-k]$.

Representation Note: Why $y^{(i)}[n], e^{(i)}[n]$ Depend on i, and When $x^{(i)}[n], d^{(i)}[n]$ Are Used

1. Why $y^{(i)}[n]$ and $e^{(i)}[n]$ carry the iteration index (i):
The filter output $y^{(i)}[n] = \sum_{k=0}^{M-1} w_k^{(i)} x[n-k]$ and estimation error $e^{(i)}[n] = d[n] - y^{(i)}[n]$ **depend directly on the adaptive weights $\mathbf{w}^{(i)}$**. Because the weights adapt from iteration to iteration $(i \to i+1)$, the output and error change dynamically at every learning step even when evaluated on the same sample n. Writing $y^{(i)}[n]$ and $e^{(i)}[n]$ explicitly tracks this progressive learning process.

2. When are $x^{(i)}[n]$ and $d^{(i)}[n]$ used? (Validity and Physical Context)

- **Online Sample-by-Sample LMS (Single Continuous Stream):** Here, $x[n]$ and $d[n]$ are generated by external physical processes (e.g., continuous sensor/microphone signals). Since discrete time increments with each sample step $(n = i)$, the input at sample n is fixed and independent of the filter's learning iteration. Therefore, writing $x^{(i)}[n]$ or $d^{(i)}[n]$ in standard single-stream LMS is **redundant**, and standard notation uses simply $x[n]$ and $d[n]$.

- **When $x^{(i)}[n]$ and $d^{(i)}[n]$ ARE Valid and Standard in DSP:** When the input or desired data sequences themselves change with each iteration i, using $x^{(i)}[n]$ and $d^{(i)}[n]$ is mathematically exact. Two major DSP formulations use this:

  1. **Representation 1: Block / Frame-Based Processing (Block LMS):** A continuous signal is split into consecutive blocks (frames) of length L. Here, i is the **block/frame iteration index** $(i = 0, 1, 2, \dots)$, and n is the **sample index within that block** $(n = 0, 1, \dots, L-1)$. The input and desired data change with each block iteration:
  $$x^{(i)}[n] = x[i \cdot L + n], \qquad d^{(i)}[n] = d[i \cdot L + n]. \tag{13}$$
  The filter computes $y^{(i)}[n] = \sum_{k=0}^{M-1} w_k^{(i)} x^{(i)}[n-k]$ and error $e^{(i)}[n] = d^{(i)}[n] - y^{(i)}[n]$ over the block, and the weights $\mathbf{w}^{(i+1)}$ update once per frame.
  2. **Representation 2: Multi-Trial / Repetitive Sweep Processing (Ensemble LMS):** In biomedical signal processing (e.g., EEG/ECG evoked responses), radar/sonar chirps, and machine learning epochs, an experiment is repeated across multiple trials. Here, i is the **trial / sweep / run number** $(i = 1, 2, \dots)$, and n is the **elapsed time relative to the stimulus trigger** $(n = 0, 1, \dots, T-1)$. Because background noise $v^{(i)}[n]$ and physical channel conditions vary from trial to trial:
  $$x^{(i)}[n] = s[n] + v^{(i)}[n], \qquad d^{(i)}[n] = \text{Target response in trial } i. \tag{14}$$
  The input and desired waveforms genuinely differ across iterations i due to trial-specific noise realizations.

4.2 4.2 Error and cost function

$$e^{(i)}[n] = d[n] - y^{(i)}[n] \tag{15}$$

Mean-square-error cost (with fixed $\mathbf{w}$):

$$J(\mathbf{w}) = E[e^2[n]] = E[d^2[n]] - 2\mathbf{w}^T\mathbf{p} + \mathbf{w}^T\mathbf{R}\,\mathbf{w} \tag{16}$$

where $\mathbf{R} = E[\mathbf{x}[n]\mathbf{x}^T[n]]$ is the input autocorrelation matrix and $\mathbf{p} = E[d[n]\,\mathbf{x}[n]]$ is the cross-correlation vector.

Wiener solution. J is a quadratic bowl in $\mathbf{w}$. Its gradient is

$$\nabla J = -2\mathbf{p} + 2\mathbf{R}\mathbf{w}. \tag{17}$$

Setting $\nabla J = \mathbf{0}$ gives the normal (Wiener–Hopf) equations

$$\mathbf{R}\,\mathbf{w}^* = \mathbf{p} \implies \mathbf{w}^* = \mathbf{R}^{-1}\mathbf{p}. \tag{18}$$

Computing this directly requires knowing $\mathbf{R}$ and $\mathbf{p}$ and inverting $\mathbf{R}$ — unrealistic in real time, hence LMS.

4.3 4.3 The Widrow–Hoff LMS Algorithm

Replace the true gradient by an instantaneous estimate from the current sample n:

$$\hat{\nabla} J^{(i)}[n] = \frac{\partial (e^{(i)}[n])^2}{\partial \mathbf{w}} = 2e^{(i)}[n]\frac{\partial e^{(i)}[n]}{\partial \mathbf{w}} = -2e^{(i)}[n]\,\mathbf{x}[n]. \tag{19}$$

Stepping in the negative gradient direction (absorbing the factor 2 into step size μ), the weight vector updates from iteration i to $i+1$:

$$\boxed{\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} + \mu\, e^{(i)}[n]\,\mathbf{x}[n]} \tag{20}$$

In scalar form, updating each individual tap weight $k \in \{0, 1, \ldots, M-1\}$:

$$\boxed{w_k^{(i+1)} = w_k^{(i)} + \mu\, e^{(i)}[n]\,x[n-k]} \tag{21}$$

- **Textbook notation equivalence:** In classical texts where iteration is denoted as n instead of i, this equation is written as $\mathbf{w}[n+1] = \mathbf{w}[n] + \mu e[n]\mathbf{x}[n]$ and $w_k[n+1] = w_k[n] + \mu e[n]x[n-k]$. Introducing i clarifies that i is the iteration count, n is discrete time, and k is the tap index.
- **Convention note:** Lecture 26 wrote $\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} + 2\mu\, e[n]\mathbf{x}[n]$. The two forms are identical with $\mu_{27} = 2\mu_{26}$. Always check which convention an exam question uses.
- With this convention, mean convergence requires $0 < \mu < 2/\lambda_{\max}(\mathbf{R})$, and the practical sufficient condition is $0 < \mu < 2/(MP_x)$, since $\lambda_{\max} \le \mathrm{Tr}(\mathbf{R}) = MP_x$.



Figure 3: LMS system identification of a 4-tap FIR system with three step sizes (convention $\mathbf{w} \leftarrow \mathbf{w} + 2\mu e\mathbf{x}$): large μ converges fast but leaves more steady-state error; small μ is slow but accurate.

4.4 4.4 Algorithmic Pseudocode (LMS Adaptive FIR Filter)

The following pseudocode outlines the complete sample-by-sample LMS implementation, making the separation between the iteration loop (i), tap index (k), and sample time (n) explicit:

```
================================================================================
ALGORITHM: Sample-by-Sample LMS Adaptive FIR Filter
================================================================================
Inputs:
  x[n]       : Input signal stream of length N_samples (n = 0, ..., N - 1)
  d[n]       : Desired target stream of length N_samples (n = 0, ..., N - 1)
  M          : Number of filter taps (filter order = M - 1)
  mu         : Step-size parameter (learning rate)
Outputs:
  y[n]       : Adaptive filter output stream
  e[n]       : Estimation error stream
  w_k^(end)  : Final converged tap weights

1. INITIALIZATION:
   For k = 0 to M - 1:
       w[k]      = 0.0      // Initial tap weight w_k^(0) = 0 for tap k
       buffer[k] = 0.0      // Delay line buffer: holds [x[n], ..., x[n-M+1]]

2. ADAPTATION LOOP (Iteration index i = 0, 1, 2, ..., N_samples - 1):
   For i = 0 to N_samples - 1:
       n = i                // Current time sample matches iteration index

       // Step A: Shift delay line buffer & insert incoming sample x[n]
       For k = M - 1 down to 1:
           buffer[k] = buffer[k - 1]
       buffer[0] = x[n]     // buffer[k] now contains delayed sample x[n-k]

       // Step B: Transversal Filter Output (Summation over tap index k)
```

```
        y[n] = 0.0
        For k = 0 to M - 1:
            y[n] = y[n] + ( w[k] * buffer[k] )

        // Step C: Instantaneous Estimation Error
        e[n] = d[n] - y[n]   // Computes error e^(i)[n]

        // Step D: LMS Tap Weight Update (Transition: iteration i -> i + 1)
        For k = 0 to M - 1:
            // w_k^(i+1) = w_k^(i) + mu * e^(i)[n] * x[n-k]
            w[k] = w[k] + ( mu * e[n] * buffer[k] )

    End For
==========================================================================
```

---

5 5. WORKED EXAMPLE 27.2 – TWO LMS ITERATIONS

Problem: Given initial weight vector at iteration $i = 0$: $\mathbf{w}^{(0)} = [0,\ 0]^T$ ($M = 2$ taps, $k \in \{0, 1\}$), step size $\mu = 0.1$, and update rule $w_k^{(i+1)} = w_k^{(i)} + \mu e^{(i)}[n]x[n-k]$, process the two input/desired pairs:

$$\mathbf{x}[0] = [x[0],\ x[-1]]^T = [1.0,\ 0.5]^T, \quad d[0] = 2.0;$$

$$\mathbf{x}[1] = [x[1],\ x[0]]^T = [0.5,\ 1.0]^T, \quad d[1] = 1.5.$$

Iteration $i = 0$ (Processing sample $n = 0$):

$$y^{(0)}[0] = w_0^{(0)}x[0] + w_1^{(0)}x[-1] = (0)(1.0) + (0)(0.5) = 0 \tag{22}$$

$$e^{(0)}[0] = d[0] - y^{(0)}[0] = 2.0 - 0 = 2.0 \tag{23}$$

$$\text{Tap } k = 0: \quad w_0^{(1)} = w_0^{(0)} + \mu\, e^{(0)}[0]\, x[0] = 0 + (0.1)(2.0)(1.0) = \mathbf{0.20} \tag{24}$$

$$\text{Tap } k = 1: \quad w_1^{(1)} = w_1^{(0)} + \mu\, e^{(0)}[0]\, x[-1] = 0 + (0.1)(2.0)(0.5) = \mathbf{0.10} \tag{25}$$

The updated weight vector for iteration $i = 1$ is $\mathbf{w}^{(1)} = [0.20,\ 0.10]^T$.

Iteration $i = 1$ (Processing sample $n = 1$):

$$y^{(1)}[1] = w_0^{(1)}x[1] + w_1^{(1)}x[0] = (0.20)(0.5) + (0.10)(1.0) = 0.20 \tag{26}$$

$$e^{(1)}[1] = d[1] - y^{(1)}[1] = 1.5 - 0.20 = \mathbf{1.30} \tag{27}$$

$$\text{Tap } k = 0: \quad w_0^{(2)} = w_0^{(1)} + \mu\, e^{(1)}[1]\, x[1] = 0.20 + (0.1)(1.30)(0.5) = \mathbf{0.265} \tag{28}$$

$$\text{Tap } k = 1: \quad w_1^{(2)} = w_1^{(1)} + \mu\, e^{(1)}[1]\, x[0] = 0.10 + (0.1)(1.30)(1.0) = \mathbf{0.230} \tag{29}$$

The updated weight vector for iteration $i = 2$ is $\mathbf{w}^{(2)} = [0.265,\ 0.230]^T$. The weights adjust along the gradient to progressively minimize output error.

---

6 6. FACULTY INSIGHTS, PITFALLS & EXAM TRAPS

- **Index Confusion (n vs. k vs. i):** The most common exam trap is mixing up the filter tap index k ($0 \le k \le M-1$) with the discrete time n or iteration count i. In $w_k^{(i)}$, k

chooses the physical tap multiplier, while i indexes the training iteration. When updating tap k, always multiply the error $e[n]$ by the sample present at tap k, which is $x[n-k]$ (not $x[n]$ or $x[k]$!).

- **Forgetting the zeros at $z=-1$:** the filter then has fewer zeros than poles and the high-frequency attenuation is wrong. Always compute $N-M$.
- **Forgetting K_d:** the unscaled $H''(1)$ is ≈ 231.75, a gain of 47 dB!
- **Wrong direction of the update:** the update is $+\mu e\mathbf{x}$ because the gradient estimate is $-2e\mathbf{x}$.
- **Mixing 2μ and μ conventions** (see §4.3).
- **Using $x[n]$ vs. $d[n]$ in the update:** LMS multiplies the *error* by the *input vector*; $d[n]$ only enters through $e[n]$.

Looking ahead (Lecture 28): The LMS equation is, symbol for symbol, the delta rule used to train a single artificial neuron. The next lecture maps DSP onto machine learning systematically.

---

7 7. QUICK REVISION CHEAT-SHEET

Topic	Formula	Note
Pole mapping	$p_k \mapsto e^{p_k T}$	stable $\rightarrow$ inside unit circle
Zeros at ∞	$(1+z^{-1})^{N-M}$	$s=\infty \leftrightarrow z=-1$
DC gain match	$K_d = H_a(0)/H''(1)$	$K_d = 0.004315$ in Ex. 27.1
Index distinction	$k=$ tap, $n=$ time, $i=$ iteration	Avoids variable overloading
Filter Output	$y^{(i)}[n] = (\mathbf{w}^{(i)})^T\mathbf{x}[n] = \sum_{k=0}^{M-1} w_k^{(i)} x[n-k]$	Output depends on weights at step i
Estimation Error	$e^{(i)}[n] = d[n] - y^{(i)}[n]$	Drives weight updates at step i
Signal context	$x^{(i)}[n], d^{(i)}[n]$ used in Block/Trial LMS	Redundant in sample-by-sample ($n=i$)
Cost function	$J = E[d^2] - 2\mathbf{w}^T\mathbf{p} + \mathbf{w}^T\mathbf{R}\mathbf{w}$	Quadratic error bowl
Wiener solution	$\mathbf{w}^* = \mathbf{R}^{-1}\mathbf{p}$	Optimal MMSE solution
LMS Vector Update	$\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} + \mu e^{(i)}[n]\mathbf{x}[n]$	Iteration step $i \rightarrow i+1$
LMS Tap Update	$w_k^{(i+1)} = w_k^{(i)} + \mu e^{(i)}[n]x[n-k]$	Tap k updated by delayed sample $x[n-k]$